

Preference Optimization for Language Models

Bridging Offline and Online DPO Through On-Policy Data
Mixing

Issey Sone

April 2026

Table of Contents

1. Introduction and Motivation

RLHF and PPO

DPO

2. Related Work

Semi-online DPO

3. Experiment and Results

Proposed Methodology

Results

Future Directions

4. References

Why Alignment matters?

Key Issue:

Training Objective $\neq$ Human Preferences

- Modern language models trained with MLE(Maximum Likelihood Estimator) on sentences taken from the internet.
- While this can produce strong generative abilities, it may not be the desirable behavior.

Examples of Misalignment:

1. hallucinations
2. toxic or harmful outputs
3. ignoring the user task

A major breakthrough was in the paper Training language Models to Follow Instructions with Human Feedback (Ouyang et al. 2022).

Pipeline:

1. Pretrain LM (MLE)
2. Collect Human Preference comparisons
3. Train reward model
4. Optimize policy using RL (PPO)

Major finding: A 1.3B RLHF model outperformed a 175B GPT-3 model in human preference evaluation.

RLHF and PPO (cont.)

PPO optimizes the following equation:

$$\max_{\pi} \mathbb{E}_{x, y \sim \pi} [R(x, y)] - \beta KL(\pi \| \pi_{ref})$$

where

- R = Reward model trained on human comparisons
- KL term keeps the model near the pretrained distribution

What PPO has allowed

PPO has allowed models to optimize non-differentiable human preferences, and learn complex behavioral objectives.

Limitations of PPO-Based RLHF

Training instability

Sensitive to hyperparameters such as KL regularization and reward scaling. Policy updates can become noisy or unstable during optimization.

High computational cost

PPO requires repeated on-policy sampling and rollout generation. Training typically involves several components, including the policy, reward, value, and reference models.

Reward hacking

The model can learn to exploit imperfections in the reward model rather than genuinely improve alignment.

Direct Preference Optimization (DPO)

Rafailov et al. 2024 showed that RLHF can be rewritten as a supervised learning objective. We can "directly" optimize preferences, so if we have preference data $y_w \succ y_l$. DPO optimizes the following:

$$\mathcal{L}_{\text{DPO}} = \log \sigma(\beta[\log \pi(y_w|x) - \log \pi(y_l|x)])$$

Benefits of DPO

- No separate Reward model is needed
- Stable training
- Cheaper, and easier to implement

Extensions of DPO

IPO¹

- *Conservative updates*

$$\mathcal{L}_{\text{IPO}} = -\log \sigma(\beta[\Delta - \tau])$$

¹ Azar et al. (2023)

KTO²

- *Asymmetric preferences*

$$\mathcal{L}_{\text{KTO}} = -\log \sigma(\beta\Delta) - \lambda \log \sigma(-\beta\Delta)$$

² Ethayarajh et al. (2024)

ORPO³

- *Odds-ratio objective*

$$\mathcal{L}_{\text{ORPO}} = -\log \frac{\pi_{\theta}(y_w | x)}{\pi_{\theta}(y_l | x)}$$

³ Hong, Lee, and Thorne (2024)

SimPO⁴

- *Minimal DPO*

$$\mathcal{L}_{\text{SimPO}} = -\log \sigma(\beta\Delta)$$

⁴ Meng, Xia, and Chen (2024)

where $\Delta = \log \pi_{\theta}(y_w | x) - \log \pi_{\theta}(y_l | x)$

What's Known

Performance differences between online and offline RL algorithms have been well established in previous literature (Levine et al. 2020).

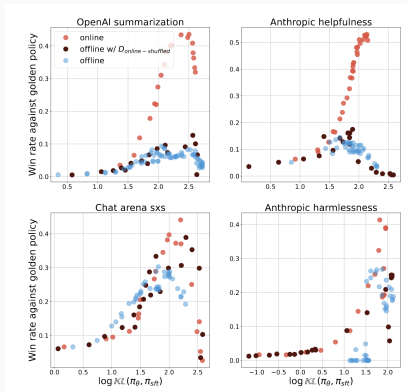


Figure 1: Win rate vs. KL budget across four alignment datasets (Tang et al. 2024).

Why Offline Methods underperform

Understanding the performance gap between online and offline alignment algorithms(Tang et al. 2024) found:

- The gap between online and offline methods is *not* explained by data quality, data coverage, the contrastive loss function, or reward classification accuracy, each was tested and ruled out individually.
- Giving offline methods access to the same data online methods generated (just shuffled) does not close the gap.

Key Advantage:

The core advantage in online methods is that the model trains on data it generated at each point in training, this creates a feedback loop between policy improvement and data generation that offline methods fundamentally lack.

Semi-online methods

Key Idea:

Instead of sampling on-policy data at every training step, periodically sync the data-generating policy with the training policy every s steps.

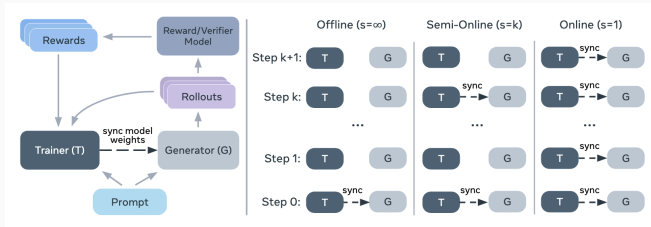


Figure 2: Visualization of a single training pipeline. The sync frequency s controls how often the training policy's weights are copied to the data-generating policy. (Lanchantin et al. 2025)

Bridging Offline and Online Reinforcement Learning for LLMs

Experimental setup:

- Model: Llama-3.1-8B-Instruct
- Two domains tested: non-verifiable (chat) and verifiable (math).
- Compare on-policy GRPO against offline, semi-online, and online DPO by varying the sync period s .

Key Findings

- Big performance jumps come from offline to semi-online, not as big of a jump from semi-online to fully online.

Training method	Math500	NuminaMath	AMC23
Seed (Llama-3.1-8B-Instruct)	47.4 (1.6)	33.9 (0.6)	23.7 (5.2)
Offline DPO ($s = \infty$)	53.7 (1.6)	36.4 (0.6)	28.8 (7.0)
Semi-online DPO ($s = 100$)	58.9 (1.2)	39.3 (0.4)	35.1 (5.3)
Semi-online DPO ($s = 10$)	57.2 (1.1)	39.4 (0.5)	31.4 (4.3)
Online DPO ($s = 1$)	58.7 (1.2)	39.6 (0.5)	32.9 (5.2)
GRPO	58.1 (1.3)	38.8 (0.5)	33.6 (5.1)

Figure 3: test accuracy (std error) for Math500, NuminaMath, and AMC23 (Lanchantin et al. 2025)

Proposed Methodology

Research Question

If fresh on-policy samples improve DPO training, how much fresh data is actually needed to recover most of the benefit of fully online training?

The experiment aims to explore two knobs simultaneously:

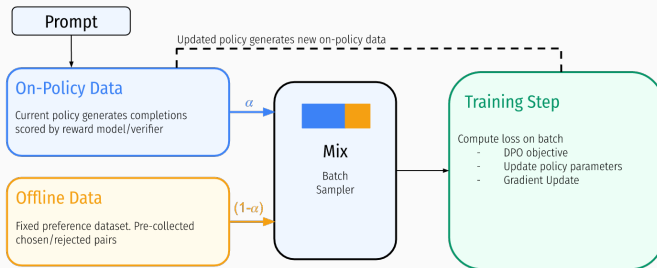
1. s : sync period, how often the generator weights sync with trainer
2. α : Fraction of each batch from on-policy samples

At each training step, the batch is composed as:

$$D_{\text{batch}} = \alpha D_{\text{fresh}} + (1 - \alpha) D_{\text{offline}}$$

where $\alpha \in [0, 1]$

Single Training Step: Blending On-Policy and Offline Data



Experiment Setup

- Model: Qwen-2.5-1.5B-Instruct fine-tuned with LoRA
- Task: GSM8K (verifiable math, binary reward)
- Training: DPO, 150 steps, batch size 4, $\beta = 0.1$
- Evaluation: 150 GSM8K test samples, 2 seeds
- 3 sync periods(s) [1, 10, 999], and 3 fraction values (α) [0, 0.5, 1]

Training Method	α	Best Acc. (%)	Gen. Cost	Wall Time (hr)
Seed (Qwen2.5-1.5B-Instruct)	–	3.3	0	–
Online DPO ($s=1$)	0.0	27.0	0	1.3
	0.5	35.3	2,400	3.7
	1.0	32.3	4,800	6.3
Semi-online DPO ($s=10$)	0.0	28.7	0	1.3
	0.5	33.3	2,400	3.6
	1.0	31.7	4,800	6.5
Offline DPO ($s=\infty$)	0.0	25.3	0	1.2
	0.5	28.0	2,400	4.84
	1.0	32.3	4,800	8.69

Table 1: Best GSM8K Accuracy by Configuration.

Training and Loss Curve

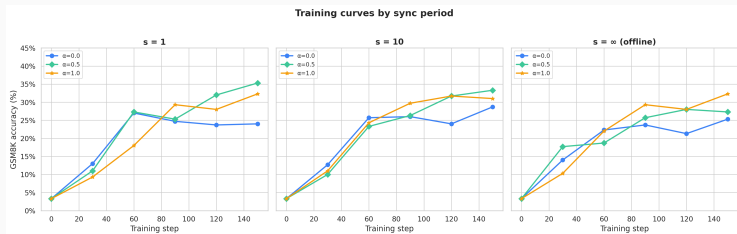


Figure 4: Training Curves for various sync periods

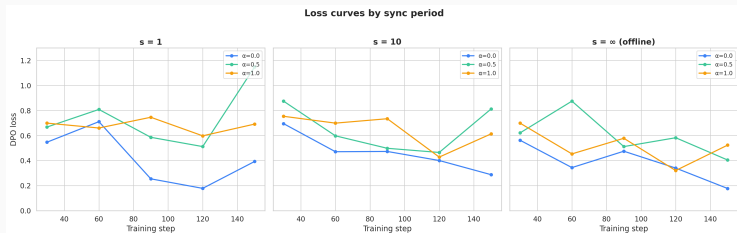


Figure 5: Loss Curves for various sync periods

Effect of fresh data

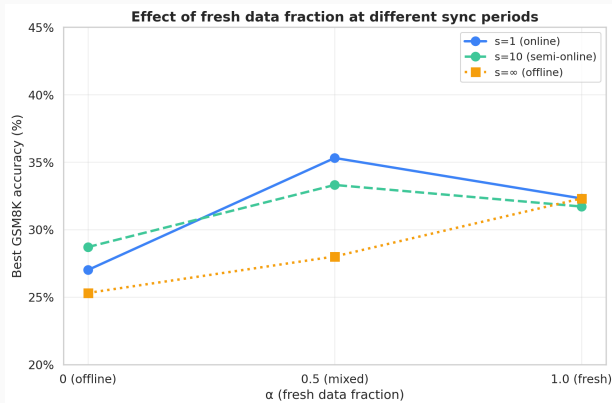


Figure 6: Effect of Fresh data at different sync periods

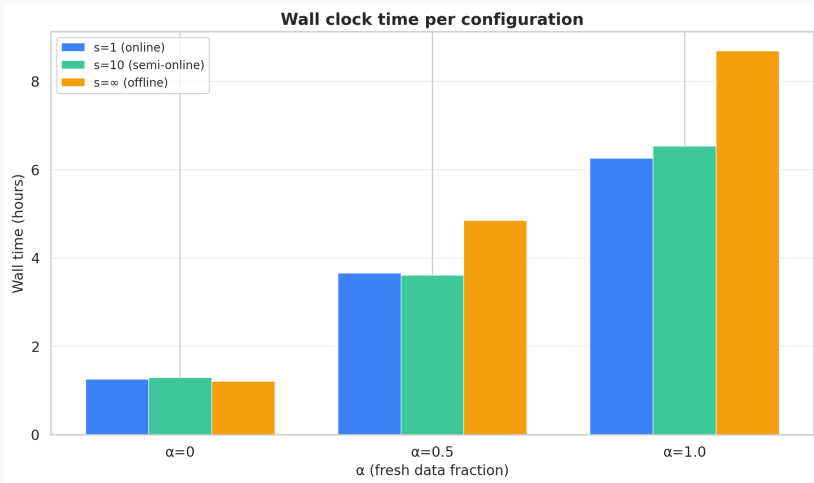


Figure 7: Wall time of each configuration

Caveats of this experiment

1. **Scale gap:** Original paper used Llama-3.1-8B with *full* fine-tuning for 1500 steps and batch size of 64. (Qwen-2.5-1.5B with LoRA for 150 steps and batch size of 4).
2. **Offline Buffer:** The buffer contained only 60 preference pairs. With batch size 4 and 150 steps, the $\alpha = 0$ configs saw each pair around 10 times on average, causing overfitting. (This makes it look artificially worse than it would with a larger buffer).
3. **Evaluation Size:** 150 test samples with 2 seeds gives a standard error of roughly $\pm 3 - 4\%$. (The $\alpha = 0.5$ advantage over $\alpha = 1$ is suggestive but not statistically significant at this sample size).
4. **Single task:** The experiment only looked at 1 verifiable task.

Overall Findings

Research Question

If fresh on-policy samples improve DPO training, how much fresh data is actually needed to recover most of the benefit of fully online training?

- **Regularization.** Pure offline training plateaus as the model overfits on the offline buffer. Adding 50% of data pushes accuracy higher across all sync periods.
- **Half-batches.** $\alpha = 0.5$ matched or outperformed $\alpha = 1.0$ at half the generation cost. The mixed regime gets stable gradient signal from the offline pairs and diversity from the fresh ones.
- **Sync period.** $s = 1$ and $s = 10$ produced very similar results for the GSM8K task. Very similar findings to Lanchantin et al. 2025

- **Increase Scale:** Increasing the batch size, offline buffer size, and look at more tasks/harder benchmarks will further verify results from this experiment.
- **Finer resolution for α :** Vary up various values of α more. Try $\alpha \in [0, 1]$ with step size of 0.1.
- **Adaptive α schedule:** Instead of fixing α for the entire training, start with a high α and go towards a lower α in the end or vice versa.



Azar, Mohammad Gheslaghi et al. (2023). **A General Theoretical Paradigm to Understand Learning from Human Preferences**. arXiv: 2310.12036 [cs.AI]. URL: <https://arxiv.org/abs/2310.12036>.



Ethayarajh, Kawin et al. (2024). **KTO: Model Alignment as Prospect Theoretic Optimization**. arXiv: 2402.01306 [cs.LG]. URL: <https://arxiv.org/abs/2402.01306>.



Hong, Jiwoo, Noah Lee, and James Thorne (2024). **ORPO: Monolithic Preference Optimization without Reference Model**. arXiv: 2403.07691 [cs.CL]. URL: <https://arxiv.org/abs/2403.07691>.



Lanchantin, Jack et al. (2025). **Bridging Offline and Online Reinforcement Learning for LLMs**. arXiv: 2506.21495 [cs.CL]. URL: <https://arxiv.org/abs/2506.21495>.



Levine, Sergey et al. (2020). **Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems**. arXiv: 2005.01643 [cs.LG]. URL: <https://arxiv.org/abs/2005.01643>.



Meng, Yu, Mengzhou Xia, and Danqi Chen (2024). **SimPO: Simple Preference Optimization with a Reference-Free Reward**. arXiv: 2405.14734 [cs.CL]. URL: <https://arxiv.org/abs/2405.14734>.



Ouyang, Long et al. (2022). **Training language models to follow instructions with human feedback**. arXiv: 2203.02155 [cs.CL]. URL: <https://arxiv.org/abs/2203.02155>.



Rafailov, Rafael et al. (2024). **Direct Preference Optimization: Your Language Model is Secretly a Reward Model**. arXiv: 2305.18290 [cs.LG]. URL: <https://arxiv.org/abs/2305.18290>.



Tang, Yunhao et al. (2024). **Understanding the performance gap between online and offline alignment algorithms**. arXiv: 2405.08448 [cs.LG]. URL: <https://arxiv.org/abs/2405.08448>.